Lab 5 — Create a Linux VM with the Azure CLI

In this lab you provision an Ubuntu virtual machine entirely from the command line, open a port, SSH in, install the NGINX web server, check its power state, and cleanly delete it — experiencing the CLI as a full alternative to the portal. This maps to the AZ-900 skill areas **Describe Azure architecture and services** (virtual machines, IaaS) and **Describe Azure management and governance** (Azure CLI as a management tool).

Run in Azure Cloud Shell — https://shell.azure.com

Step 1 — Ensure the resource group exists

```
az group create --name rg-az900-labs --location southeastasia --tags course=az900
```

Idempotent — succeeds whether or not the group already exists from earlier labs, so this lab is self-contained.

Step 2 — Create the Linux VM

```
az vm create \
  --resource-group rg-az900-labs \
  --name vm-az900-linux01 \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --admin-username azureuser \
  --generate-ssh-keys
```

One command creates the VM plus its VNet, subnet, public IP, NIC and NSG (defaults are generated when not specified). `--generate-ssh-keys` creates a key pair in `~/.ssh` (or reuses an existing one) and installs the public key — no password needed. After 1–2 minutes it returns JSON; note the `publicIpAddress` field. `Standard_B1s` (1 vCPU, 1 GiB) is the cheapest general-purpose size — perfect for a lab web server.

Step 3 — Open port 80 for web traffic

```
az vm open-port --resource-group rg-az900-labs --name vm-az900-linux01 --port 80
```

Adds an allow rule for TCP 80 to the VM's network security group — the CLI equivalent of the portal NSG rule you created in Lab 4. SSH (22) was already opened by `az vm create` for Linux images.

Step 4 — Get the VM's IP addresses

```
az vm list-ip-addresses --resource-group rg-az900-labs --name vm-az900-linux01 -o
table
```

Shows the VM's public IP and private IP in a table. Copy the **PublicIPAddresses** value — you will use it for SSH and the browser test. Store it in a variable to avoid retyping:

```
IP=$(az vm list-ip-addresses -g rg-az900-labs -n vm-az900-linux01 \
  --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv)
echo $IP
```

Step 5 — SSH into the VM from Cloud Shell

```
ssh azureuser@$IP
```

Type `yes` at the host-key prompt. You land on the Ubuntu shell (`azureuser@vm-az900-linux01:~$`) — Cloud Shell already holds the private key generated in Step 2, so login is passwordless. This is the Linux equivalent of the RDP connection you made in Lab 4.

Step 6 — Install NGINX inside the VM

```
sudo apt update && sudo apt install -y nginx
```

Refreshes the package index and installs the NGINX web server; the service starts automatically on install. Verify from inside the VM:

```
curl http://localhost
```

Returns the `Welcome to nginx!` HTML — the web server is running locally.

Step 7 — Exit and test from outside

```
exit
```

Returns you to Cloud Shell. Now test the public endpoint:

```
curl http://$IP
```

Returns the same `Welcome to nginx!` page — traffic now flows internet > public IP > NSG (port 80 rule from Step 3) > VM. You can also open `http://<public-ip>` in your laptop's browser.

Step 8 — Check the VM's power state

```
az vm show --resource-group rg-az900-labs --name vm-az900-linux01 --show-details
--query powerState -o tsv
```

`--show-details` adds runtime info (power state, IPs) to the model view; expect `VM running`. Try deallocating and re-checking to see billing-relevant states:

```
az vm deallocate -g rg-az900-labs -n vm-az900-linux01
az vm show -g rg-az900-labs -n vm-az900-linux01 --show-details --query powerState -o
tsv
```

Now shows `VM deallocated` — compute billing has stopped, exactly as you observed in the portal in Lab 4.

Step 9 — Delete the VM and its attached resources

```
az vm delete --resource-group rg-az900-labs --name vm-az900-linux01 --yes
```

Deletes the VM **compute resource only** — by default the NIC, public IP, OS disk and NSG it used are left behind as **orphaned resources** that can still incur cost (disk, static IP). List what remains:

```
az resource list --resource-group rg-az900-labs -o table
```

Delete the leftovers by name (adjust names to match the table output):

```
az network nic delete -g rg-az900-labs -n vm-az900-linux01VMNic
az network public-ip delete -g rg-az900-labs -n vm-az900-linux01PublicIP
az network nsg delete -g rg-az900-labs -n vm-az900-linux01NSG
az disk list -g rg-az900-labs --query "[?contains(name,'linux01')].name" -o tsv | \
  xargs -I{} az disk delete -g rg-az900-labs -n {} --yes
```

Alternative: delete the VM in the **portal** instead, where the delete pane offers **"Delete with VM"** checkboxes for the disk, NIC and public IP in one action. Orphaned resources are a classic real-world (and exam) cost-management gotcha.

Clean up

Steps 8–9 already deallocated and deleted the VM and its attached resources. Verify nothing billable is left:

```
az resource list --resource-group rg-az900-labs -o table
```

Only resources from other labs (e.g. Lab 4's VM and VNet, if you kept them) should remain. **Keep the resource group** `rg-az900-labs` — it is reused by the remaining labs and deleted only in Lab 16.

What you learned

- One `az vm create` command provisions a complete IaaS stack — VM, VNet, NIC, public IP, NSG — with sensible defaults and SSH keys via `--generate-ssh-keys`.
- `az vm open-port` is the CLI shortcut for adding an NSG inbound rule.
- Managing a Linux VM end to end: SSH from Cloud Shell, `apt install nginx`, and verifying with `curl` inside and outside the VM.
- `az vm show --show-details --query powerState` distinguishes `VM running` from `VM deallocated` (no compute charges).
- `az vm delete` leaves orphaned NICs, IPs and disks that still cost money — clean them up explicitly, or use the portal's "Delete with VM" checkboxes.

Files in this lab folder

File	What it is
`deploy-linux-vm.sh`	Creates the Ubuntu VM, opens port 80 and prints the public IP and SSH command.
`cleanup-linux-vm.sh`	Deletes the VM **and** its leftover NIC, public IP, NSG and disk.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.